ts-drp
History at Scale

Attested Epoch Cuts v3.1 review and browser-first production design

Decision

Do not implement AEC v3.1 unchanged. Preserve its attestation, atomic transition, and adversarial discipline, but move to authenticated hard epochs so compact-peer metadata is not O(lifetime vertices).

Repository	Faolain/ts-drp-1
Baseline	bf7d3516f6ed4be97a755698b4fb3a404e04dc0f
Recommended protocol	Attested Hard Epochs (AHE) v4
Evidence	22 tests; 5,231 exhaustive DAGs; 10,000 randomized epochs; 20,000 schedules
Date	24 July 2026

Independent engineering review and executable reference

Contents

Evidence at a glance

Validated in this review

22/22 tests pass; 91.71% source-line coverage; 5,231 exhaustive admissible DAGs through seven vertices; 10,000 randomized epochs; 20,000 delivery schedules; 11,612 quorum-pair checks; 8,547 same-round QC-pair checks.

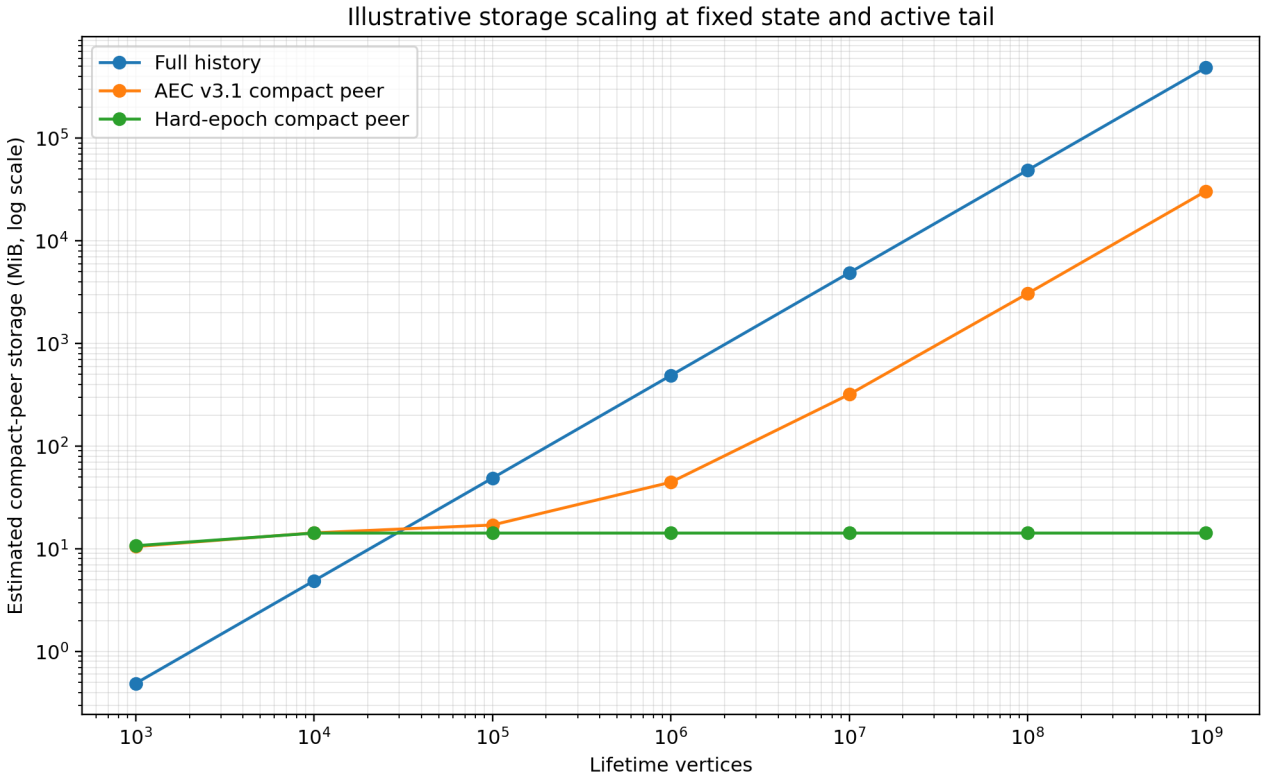

Figure 1. Illustrative compact-peer storage scaling at fixed state, active tail, and governance-change counts. Archive payload bytes are excluded.

Performance signals

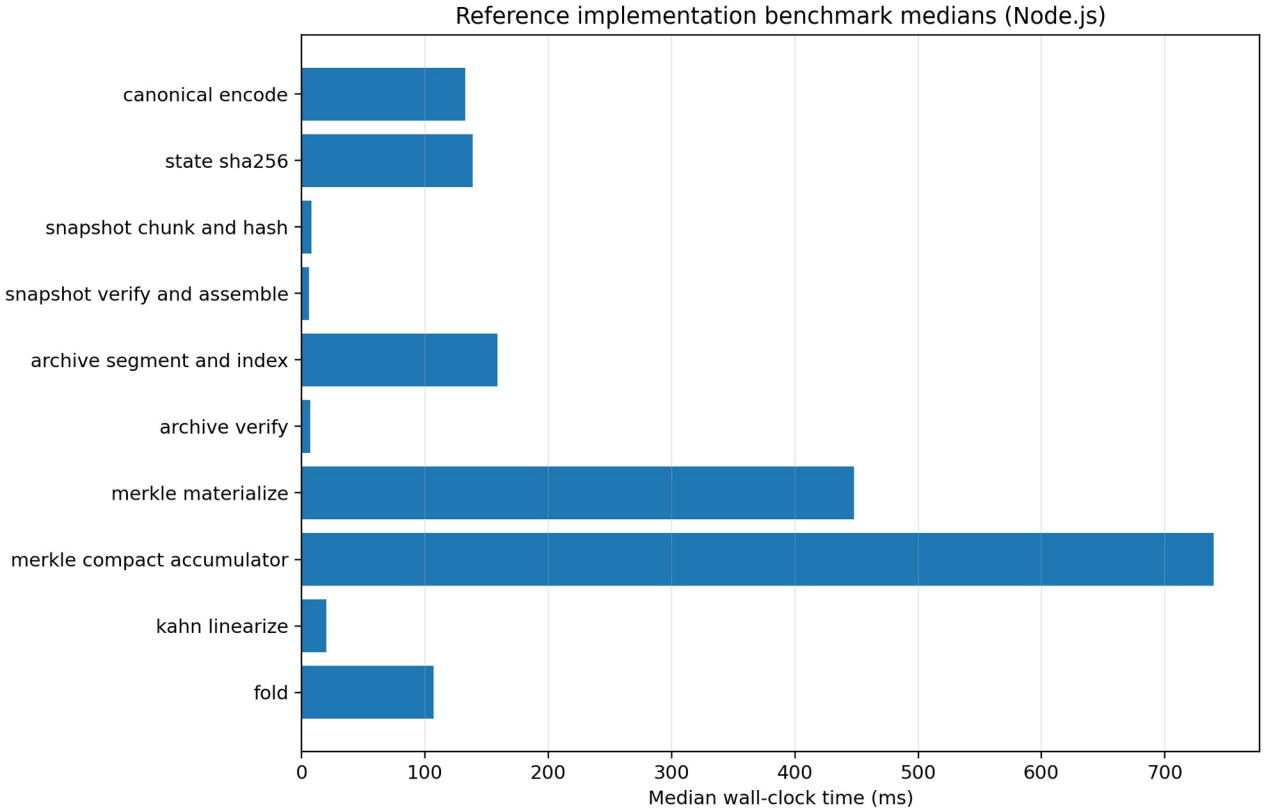

Figure 2. Reference benchmark medians on Node.js/x64. These guide Worker scheduling and are not browser guarantees.

Browser execution note

The source-level browser audit passes. The live Chromium harness was blocked before page load by the managed execution profile's URLBlocklist policy, so real Chrome/Firefox/Safari runs remain a release gate.

Executive verdict

The Attested Epoch Cuts (AEC) v3.1 plan is unusually strong as an adversarial design review. It correctly recognizes that compaction is not a local storage optimization: it changes which delayed operations can still affect the object, so it must become a deterministic protocol rule. It also identifies several existing correctness and security defects in the repository that would become load-bearing under snapshots, including cross-room vertex replay, non-canonical hashing, shallow state adoption that cannot represent deletion, non-atomic batch application, wall-clock-dependent permanent invalidity, and full-history synchronization.

AEC v3.1 should not, however, be implemented unchanged. Its mandatory exact set of every covered vertex hash costs 32 bytes per pruned vertex forever. That bounds replay and graph structures but leaves compact peers with linear lifetime metadata. At 100 million vertices, the illustrative model assigns about 2.99 GiB to an AEC compact peer even before object-store and index overhead. The same exact set is not replaceable by a Bloom, cuckoo, or XOR filter: a false positive is a semantic false rejection, so two replicas can converge differently. A Merkle root does not solve local absence queries either; it proves membership only when a proof is supplied.

AEC also retains a same-epoch tail above a synthetic root. Correctness therefore depends on an origin-independence lemma for the repository's linearization. The executable model reproduces a concrete counterexample in the legacy DFS order: contracting a valid-looking prefix changes the order of concurrent tail vertices from `[2,4,3]` to `[2,3,4]`. The current iterative DFS also mutates adjacency arrays while sorting and has a shared-descendant double-emission/overwrite hazard. Enforcing direct-dependency antichains narrows the problem, but a production protocol should not depend on a difficult contraction theorem when a cleaner boundary is available.

The recommended design is **Attested Hard Epochs**:

- every vertex authenticates its object, protocol major, epoch, and epoch anchor;
- a committed close consumes one exact dependency-closed set and carries no ordinary same-epoch tail through the root swap;
- excluded old-epoch operations become objectively stale from their signed envelope and may be rebased only by their original author;
- deterministic min-hash Kahn ordering replaces origin-sensitive DFS ordering;
- authority is latched at the epoch anchor and changes take effect at the next anchor;
- cut votes and quorum certificates are sidecar control records, not vertices inside the graph they finalize;
- snapshots are canonical, content-addressed, chunked, and staged in IndexedDB;
- an RFC 9162-style append-only Merkle history gives logarithmic continuity proofs without becoming an admission oracle;
- Discord-like message history is moved into immutable authenticated archive segments, allowing a normal joiner to fetch a hot snapshot and recent window while old history remains demand-loaded;
- the first browser profile should carry individual existing identity signatures in quorum certificates rather than adding BLS/WASM and proof-of-possession complexity prematurely.

This design is supported by an independent executable reference and finite models. The packaged evidence currently has 22/22 passing tests, 91.71% source-line coverage, 5,231 exhaustive admissible DAG checks through seven vertices, 10,000 randomized epochs, 20,000 hostile delivery schedules, 11,612 quorum-pair checks, 8,547 same-round QC-pair checks, an exact legacy-contraction counterexample, and an approximate-membership divergence counterexample. A static browser audit reports no Node built-ins, CommonJS, synchronous XHR, `eval`, web-storage dependency, wall-clock semantic decision, random semantic decision, or `JSON.stringify` hashing in the protocol core.

The reference is **production-target evidence, not a completed production integration**. Release still requires a formal pacemaker/locking model, repository-specific wire and signature adapters, crash injection against real IndexedDB implementations, a Chrome/Firefox/Safari/mobile matrix, adversarial network testing, migration rehearsal, and external security review. The supplied Chromium run was blocked before page load by the execution environment's managed `URLBlocklist=*` policy; the browser harness is included for execution in a normal secure profile.

1. Scope and method

1.1 What was reviewed

The review covered four mutually dependent layers:

1. **The current repository implementation.** The hashgraph, state manager, applier, finality plumbing, sync handlers, vertex hashing, and existing in-memory checkpoint seams were inspected at the baseline commit.
2. **AEC v3.1.** Every major mechanism was checked: stable frontiers, creator bootstrap, signer-set derivation, cut descriptors, two-phase sealing, locks, monotone rounds, evidence handoff, fold/ejection, synthetic roots, post-cut admission, joiner bootstrap, storage, migration, and the stated invariant/test matrix.

3. **The supplied Topology research.** The DRP whitepaper and protocol walkthrough were read around snapshotting, compaction, coherence, transaction re-injection, recursive proofs, and outdated-message ambiguity. The AA24 transcript and PRIBLT material were used to understand the intended browser constraints and reconciliation direction.
4. **Executable independent validation.** A browser-surface JavaScript reference and separate Python models were created. The models do not call the JavaScript implementation as an oracle.

1.2 Review standard

A mechanism was treated as production-ready only when all of the following were addressed:

- deterministic semantics under arbitrary network reordering and duplication;
- objective classification that does not depend on local time, retry count, cache residency, or probabilistic membership;
- canonical authenticated bytes with domain separation;
- crash-safe durable transitions and vote issuance;
- bounded browser memory and cooperative execution;
- explicit authority, freshness, availability, and migration assumptions;
- an executable validation path with adversarial counterexamples and fault injection.

This standard is intentionally stricter than “a snapshot can be serialized and loaded.” A snapshot is useful only if replicas agree on the state it represents, on the operations it excludes, on who was authorized to certify it, and on how future operations attach to it.

2. Current repository: why history scales poorly

2.1 Hot graph and state are lifetime-sized

The live `HashGraph` retains `vertices`, `frontier`, `forwardEdges`, and distance/cached-causality structures. Per-vertex finality state and per-vertex application/ACL snapshots add more lifetime growth. Existing checkpoints improve replay start points but do not delete graph history. The bitset causality cache is especially dangerous if activated over large graphs because its logical size is quadratic in the number of vertices.

The present checkpoint seam is still valuable. The applier already knows how to reconstruct from explicit application and ACL states and how to seek a causal barrier before replaying a suffix. These are good integration points for a durable epoch transition, but the current checkpoint is not a certified protocol boundary.

2.2 Sync is full-inventory and full-history

A sync request sends `object.vertices.map(v => v.hash)`. The responder creates a set of all local vertices, linearly searches each requested hash with `find`, and returns the difference. This is at least linear in history for every probe and can become quadratic. A fresh joiner ultimately receives the complete graph, with no epoch-scoped snapshot path, no resumable content-addressed chunks, and no admission budget on serving history.

PRIBLT or another set-reconciliation scheme can improve **bounded active-tail diffing**, especially when peers have similarly sized sets and the symmetric difference is small. It cannot replace snapshot transfer, archive paging, or semantic compaction. Its cost is proportional to set difference, not to the information content of a large state blob.

2.3 Restart loses the room

The object store is memory-first; browser restart requires network reconstruction. Compaction requires durable local state even for non-signers, and it requires stronger durability for signers because emitting two conflicting votes after a crash or multi-tab race can violate safety.

2.4 Existing defects must be fixed before snapshots become authoritative

The AEC defect census is substantially correct. The highest-severity issues are:

Finding	Why it matters	Required action
Vertex hash omits `objectId` and protocol domain	A signed chain can be replayed across rooms created by the same identity; snapshots would certify the wrong authority history	New protocol-major preimage including object, protocol, epoch, and anchor
`JSON.stringify` is the hash codec	Object insertion order and unsupported values can change bytes; two wire stacks do not share one canonical definition	Frozen deterministic codec; reject non-canonical values and encodings
State adoption uses assignment	Deleted keys survive; replay-from-snapshot can differ from replay-from-genesis	Delete-then-restore replacement and explicit snapshot schema
Replica-local `context` is enumerable	Caller/replica details can poison state digests	Exclude local context and reattach after import

Finding	Why it matters	Required action
Batch apply mutates graph before final adoption	A plain authorization/application error can leave graph and live state out of sync	Stage entire batch/fold and adopt once
Wall-clock future checks enter permanent invalid memory	A vertex can be invalid at one arrival time and valid later; delivery schedule affects graph	Quarantine clock anomalies; only deterministic failures are terminal
Missing dependency paths sometimes skip silently	A compacted peer may calculate causality or order with incomplete ancestry	One central fail-closed classifier and graph API
Legacy DFS is origin- and structure-sensitive	Synthetic-root contraction can change operation order	Deterministic Kahn ordering and strict dependency invariants
Full hash inventory sync	Every probe and join grows with room lifetime	Epoch summary, snapshot manifests, bounded tail reconciliation
Finality keys/signer handling is underspecified	Empty/malformed keys and missing proof-of-possession weaken aggregate-signature assumptions	Prefer existing identity signatures initially; validate key suites explicitly

These fixes are useful independently of compaction and should land first behind tests.

3. What AEC v3.1 gets right

AEC v3.1 contains many decisions that should be retained.

3.1 Compaction is a semantics contract

The central insight is correct: in open admission, a replica cannot know that no concurrent operation will ever appear. A committed boundary must therefore make some late operations invalid by a rule every replica can check. Without that rule, a full replica may accept an old operation while a compact replica lacks the ancestry needed to decide, and they can diverge.

This directly matches the supplied Topology research. The protocol walkthrough treats compaction as safe only over stable messages, forwards a computed state into a new synthetic history, and identifies outdated messages as a coherence problem rather than a compression detail. The whitepaper similarly says that re-injected transactions have lost original causal dependencies and require deterministic rewriting or another explicit policy.

3.2 Stability and certification are separate from storage

AEC correctly requires a stable/attested cut before destructive pruning. A local “I have not seen anything else” observation is not finality. The signer set is pinned from authority state, the descriptor commits to the state and history boundary, and a quorum certificate makes the transition externally verifiable.

3.3 Creator bootstrap is explicit

The plan catches the epoch-1 bootstrap circularity: no BFT signer set exists until authority/key operations have been applied. Treating cut 0 as creator-signed and explicitly assuming creator non-equivocation is honest. Invite/rendezvous pinning is the correct mitigation for cut-0 equivocation; no local algorithm can turn two valid creator-signed roots into one objective latest root.

3.4 Two-phase, value-bound votes and monotone rounds

The progression from one-phase locks to a Tendermint-style prepare/commit core is sound. A prepare vote must name the exact value/proposal. Locks change only with a sufficiently justified higher-round value. The persisted `enteredRound` rule is essential: without it, a late prepare QC from an old round can cause an honest signer to commit an old value after participating in a newer round. The independent model reproduces this retroactive fork under the old rule and blocks it with monotone rounds.

3.5 Pending is not a semantic verdict

AEC v3.1 correctly withdraws bounded-retry terminal classification. “Dependency not obtained before my local retry budget” is an availability event, not proof of invalidity. A potentially valid current-epoch child must remain semantically pending. Memory can be bounded by eviction, but retransmission must be evaluated afresh.

3.6 Atomic transition and state replacement

The proposal correctly requires a staged fold, digest verification, durable journal, one adoption boundary, and post-adoption pruning. It also correctly promotes delete-aware state replacement and replica-local context separation to preconditions.

3.7 Weak subjectivity and data availability are stated

AEC does not pretend that a certificate proves global freshness or that a hash makes bytes available. Fresh joiners need independent tips or an invite-pinned digest. Durable content needs replicas or archives. These assumptions should remain visible in product UX.

---

4. Why AEC v3.1 still does not solve compact-peer history at scale

4.1 The exact covered-hash oracle is still O(lifetime vertices)

AEC's post-cut admission asks whether an unknown dependency hash is behind a sealed cut. Because the old vertex envelope does not itself reveal its epoch/anchor, a compact peer must retain exact membership for every covered vertex. The plan therefore mandates sorted 32-byte hash segments.

That is safer than an approximate filter, but it is not bounded compaction. At one billion vertices the exact hashes alone are about 29.8 GiB. Database pages, keys, indexes, and segment metadata increase the real cost. The storage model deliberately understates full-history overhead and still shows the asymptotic problem.

A probabilistic filter is prohibited at this semantic boundary. Suppose valid current-epoch parent `x` is not yet present and a filter falsely says `x` is covered. Replica A terminally rejects child `y -> x`; replica B says pending, later receives `x`, and applies both. The replicas diverge solely because of a false positive. The included counterexample executes this scenario.

An ordinary Merkle tree also cannot answer “this unknown hash is absent” without an authenticated dictionary/non-membership construction, and a proof must be available. Making terminal validity depend on a peer supplying a proof reintroduces availability-dependent semantics.

4.2 Synthetic-root tail replay depends on a fragile origin lemma

AEC bakes `closure(F)` into state, keeps admissible same-epoch vertices above the cut, creates a synthetic root, and maps edges from baked vertices to that root. The tail must then linearize exactly as it would from the original graph. That is not true for an arbitrary deterministic topological sort; it must be proven for the exact algorithm and graph restrictions.

The current repository order is reverse-postorder DFS over sorted forward edges. It is sensitive to origin and adjacency structure. The independent exhaustive model finds this concrete graph:

```
0 = root
1 -> 0
2 -> 1
3 -> 1
4 -> {0, 2}   // direct dependencies are causally related

baked = {0,1}
tail  = {2,3,4}
legacy full tail order       = [2,4,3]
legacy contracted tail order = [2,3,4]
```

For order-sensitive reducers, these are different states. Enforcing dependency antichains rejects vertex 4 and may make a restricted lemma provable, but the production system would still carry significant proof and implementation risk. Hard epochs eliminate the retained same-epoch tail and therefore eliminate the contraction-equivalence obligation.

4.3 In-DAG seal evidence is recursively awkward

AEC places proposals/prepares/commits inside the same DAG being cut, then needs ancestry rules and a next-cut evidence witness to prevent the certificate evidence from being ejected. This is carefully patched in v3.1, but it is avoidable complexity.

Seal messages are control-plane evidence. They should have their own signed namespace, durable vote store, gossip rules, and retention policy. A cut descriptor commits to a QC; the QC need not be an application vertex. Sidecar evidence removes circular ancestry questions and makes archive/compact retention clearer.

4.4 ACL and descriptor chains remain lifetime-linear

AEC's joiner replays ACL history and verifies a descriptor for every cut. Even if each descriptor is small, message-frequency epochs can make the chain large. Authority changes are normally much rarer than messages, so trust continuity should be represented as **authority handoffs**, while data snapshots can be checkpointed under the same authority root. Existing peers verify append-only history consistency; fresh peers verify a recent authority certificate plus weak-subjectivity tip.

4.5 Snapshotting does not shrink semantically retained chat

A game often collapses many inputs into a small world state. Chat messages are the product. If every message remains inside the snapshot, a one-year room still requires a one-year snapshot download. Compaction must distinguish hot state from historical application records.

For a Discord-like product, old messages should be canonical immutable archive segments under an authenticated index. The hot snapshot contains room/ACL state, recent messages, current threads, edit/tombstone overlays, and the archive root. Old segments are fetched and verified on demand.

4.6 Browser cryptography and durable voting need a simpler first profile

BLS aggregation saves bytes but adds a WebAssembly/native-library dependency, proof-of-possession requirements, key validation, key-rotation history, and more audit surface. Browser signer sets are likely small. Individual Ed25519/secp256k1 identity signatures in a QC are easier to inspect and integrate. Aggregation can be a later negotiated suite.

Multi-tab safety cannot rely on Web Locks alone. Web Locks coordinate cooperating contexts, but the correctness boundary must be a unique IndexedDB record written atomically before gossip. The persisted exact vote bytes are rebroadcast after restart; they are never regenerated from mutable state.

---

5. Recommended protocol: Attested Hard Epochs v4

5.1 Core semantic boundary

Every ordinary vertex authenticates:

```
{
  kind: "drp-vertex",
  protocolMajor,
  objectId,
  epoch,
  anchor,
  author,
  logicalTime,
  dependencies,
  operation
}
```

Dependencies are sorted, unique, current-object/current-epoch/current-anchor hashes. They form a direct antichain. The anchor is a synthetic protocol object, not a network-authored ordinary vertex.

After epoch `e` closes and anchor `A_(e+1)` is adopted:

- every signed vertex with `epoch < e+1` is objectively stale;
- every vertex claiming epoch `e+1` but another anchor is terminal;
- unknown dependencies within the current envelope are pending;
- no lifetime covered-hash database is needed.

This is the decisive change. The old message carries the proof of its age in its authenticated identity.

5.2 Exact close set; no carried ordinary tail

A proposal names a close frontier. The committed close set is its dependency closure within the epoch. Every signer obtains the complete set and independently recomputes order, state, ACL, history, archive, and snapshot digests.

No ordinary vertex from the closing epoch is reparented into the next active graph. Locally known operations outside the committed set are surfaced as excluded. The client SDK retains user intent in an outbox and asks the blueprint to rebase or expire it.

This is not silent loss. It is a deterministic finality boundary similar to a block boundary: an operation either entered the certified close set or did not. A short close barrier, frontier exchange, and bounded reconciliation reduce exclusions, but they do not define validity.

5.3 Rebase and stable application identifiers

Only the original author can re-sign an excluded operation for the new anchor. Cross-epoch semantic references use stable application IDs, not old graph hashes.

Examples:

- chat send carries a stable message ID and client operation ID; resubmission is idempotent;
- message edit references the stable message ID;

- a game input normally expires when its simulation tick/epoch closes;
- an inventory mutation can revalidate against current state or request user review;
- ACL changes trigger an early close and are re-authorized under the new anchor.

The Topology research's outdated-message ambiguity is resolved by policy rather than guessed away: old signatures remain valid evidence of who authored the old intent, but only a new signature can authorize a current-epoch operation.

5.4 Deterministic Kahn order

The canonical order is Kahn's algorithm with a min-hash priority queue. For an exact finite dependency-closed DAG:

1. in-degrees are uniquely determined;

2. the zero-in-degree set is uniquely determined at each step;

3. selecting the minimum hash is deterministic;

4. child decrements depend only on the graph, not arrival or map insertion;

5. a short output proves a cycle or missing edge and fails closed.

This removes DFS origin, mutable adjacency sorting, and recursion/stack hazards. The model randomizes insertion and delivery order and obtains identical output.

5.5 Latched authority

`ACL_e` is part of anchor `A_e`. Every ordinary operation in epoch `e` is authorized against that fixed state. ACL operations produce staged `ACL_(e+1)` and become effective only after the close.

This avoids authorization changing retroactively with linearization and lets a joiner import the certified ACL snapshot rather than replaying all ACL history. Urgent moderation actions use an early epoch close. The product should expose the delay and signer availability rather than imply an instantaneous revoke under partition.

5.6 Canonical encoding and domain separation

Use one frozen deterministic CBOR profile in production, with shortest encodings, deterministic key order, no indefinite lengths, no duplicate canonical keys, and strict schema/value restrictions. Decoders reject non-canonical bytes. RFC 8949 explicitly provides core deterministic encoding requirements; the protocol must add its JavaScript value and schema restrictions.

Every digest uses an object-type/version domain and length-delimited parts. Vertex, anchor, cut, snapshot, archive segment, Merkle leaf/node, vote, and QC domains are distinct. The reference codec tests insertion-order independence, minimal integer encodings, duplicate canonical keys, accessors, cycles, non-finite numbers, unpaired surrogates, and trailing bytes.

5.7 Sidecar seal protocol

For `n >= 4` and fewer than `n/3` Byzantine signers, quorum is `ceil(2n/3)`. Each round has a value-bound proposal, prepare votes, and commit votes. A signer persists:

- current/entered round;
- one exact prepare vote per round;
- one exact commit vote per round;
- lock digest and lock round;
- highest verified prepare QC;
- finalized commit QC.

A signer never votes below `enteredRound`. It prepares another value only with sufficient higher/equal-lock justification. The pacemaker uses monotonically increasing timeouts and authenticated round-change evidence; local timeout alone does not unlock or adjudicate operations.

CometBFT's public specification demonstrates why one vote per height/round/phase, durable anti-double-signing state, locks, and later-round proof-of-lock-change are load-bearing. The AHE finite model checks quorum intersections and the specific late-QC regression, but the production pacemaker still needs TLA+/Quint-level exploration.

5.8 Creator-trusted profile for small rooms

A four-signer BFT floor may be unrealistic for a two-person chat or transient game lobby. Such rooms can use a creator/delegated authority signature over the same cut object. They get deterministic snapshots and untrusted mirrors but not Byzantine-fault-tolerant finality. The trust mode must be visible and migration between modes must be an authenticated authority handoff.

6. Snapshot, history, and archive architecture

6.1 Canonical snapshot

The snapshot payload commits to object, protocol, epoch, anchor, schema, blueprint, application state, ACL state, and archive index root. It is canonical bytes, not a serialization of a live class instance.

Import is replacement:

1. validate manifest and resource maxima;
2. fetch chunks by digest;
3. verify each chunk and payload digest;
4. canonical-decode and schema-validate;
5. reconstruct isolated application and ACL instances;
6. verify state/ACL/archive digests and cut certificate;
7. atomically change the active epoch pointer;
8. reattach replica-local context.

6.2 Chunking and resumability

The proposed default is 128 KiB canonical chunks. The manifest records ordered digest/length pairs. Mirrors and peers may return chunks in any order; the receiver stores by digest and resumes missing chunks. Transport compression is outside consensus: the receiver decompresses and verifies the canonical bytes.

A receiver enforces maximum payload size, chunk count, individual chunk size, map/array nesting, decoded allocation, and concurrent fetches before allocating large buffers. Four to eight concurrent chunk fetches is a reasonable starting point for browser tuning, not a protocol constant.

6.3 Append-only history commitment

Committed vertex hashes are appended in canonical epoch order to an RFC 9162-style binary Merkle tree. Distinct leaf and internal-node prefixes provide domain separation. A compact peer retains tree size, root, and O(log N) compact-range peaks; archives keep leaves/internal nodes needed for proofs.

An existing peer verifies a consistency proof before accepting a later root. An archive supplies inclusion proofs for historical vertices or archive segment descriptors. This commitment supports audit continuity; it intentionally does not answer unknown-dependency absence during admission.

6.4 Discord-like archive segments

The hot snapshot should contain:

- room metadata, roles, bans, channels, configuration;
- recent message window and active threads;
- current reactions/edit/tombstone overlays;
- attachment roots and retention policy;
- stable operation-ID deduplication window;
- archive index root and recent segment descriptors.

Finalized old records are packed into immutable canonical segments. A segment descriptor commits to object, schema, ordinal/time range, record count, payload/chunk digests, and total length. Descriptors are leaves in a Merkle archive index.

A joiner normally fetches the certified hot snapshot, current active tail, and a configured recent segment window. Scrolling/searching older history fetches descriptors, inclusion proofs, and chunks from any mirror. Malicious bytes fail verification; unavailable bytes remain unavailable.

6.5 Edits, deletion, privacy, and attachments

Archived messages are referenced by stable message IDs. Later edits/deletes are new committed records or hot overlays. Periodic re-segmentation may produce a new archive root if the product needs compacted overlays.

Untrusted mirrors cannot be forced to erase copied plaintext. Privacy-sensitive rooms should encrypt segments and attachments with per-segment keys. A certified key-erasure record can make retained ciphertext unreadable to conforming clients, but cannot revoke plaintext already seen or copied.

Attachments should be independent content-addressed objects with size/type limits, chunk manifests, and optional encryption. The room state commits only to descriptors and policy.

7. Browser-first storage and execution

7.1 IndexedDB durability tiers

IndexedDB 3.0 exposes `strict` and `relaxed` durability hints. The recommended split is:

- **relaxed:** immutable chunk cache, archive cache, rebuildable indexes, unconfirmed outbox copies;
- **strict:** signer vote slots, entered-round/lock state, committed cut/QC, staged epoch metadata, and active pointer transition.

The stage/adopt protocol writes immutable chunks and manifests first, then writes a complete staged-epoch record, then changes one active pointer in a strict transaction. Cleanup of older generations happens later. Recovery selects either the old complete pointer or the new complete pointer; a mixed state is not addressable.

User agents treat durability as a hint, so the implementation must still retain rollback generations and test real crash behavior. `navigator.storage.persist()` and `estimate()` should be used to request non-evictable storage where available and to refuse downloads that exceed quota.

7.2 Multi-tab signer safety

Web Locks can reduce contention by allowing only one cooperating tab/worker to enter a signing critical section. They are not sufficient proof of safety. The authoritative operation is an IndexedDB unique insert/CAS keyed by `(objectId, epoch, round, phase, signerId)` that stores the exact signed bytes before network emission.

If a record already exists:

- identical bytes are rebroadcast;
- a different value is a local safety alarm and no second vote is emitted.

This protects same-origin tabs. Multiple devices sharing one private signer key remain a separate deployment risk and require a hardware/remote signer or coordinated signing service.

7.3 CryptoKey and key exposure

WebCrypto defines serializable `CryptoKey` objects and expects IndexedDB to be used for storage without exporting key material to JavaScript. That is preferable where the deployed signature algorithm and browser support permit it. The threat model must still treat same-origin script injection as signer compromise; strong CSP, dependency pinning, and supply-chain controls are required.

7.4 Workers and cooperative scheduling

Canonical encoding, state hashing, archive packing, full fold, and large Merkle work should run in a dedicated Worker. Loops should process bounded batches and yield/cancel. Live chat/game operations should not block on a close; new writes during the short barrier enter the next-epoch outbox.

The reference benchmark is Node/x64, not a browser guarantee. Median observations for a 1.88 MiB / 10,000-message state were approximately:

Operation	Median
Canonical encode	132.7 ms
State SHA-256	138.6 ms
Snapshot chunk + manifest	7.7 ms
Snapshot verify + assemble	6.1 ms
Ten archive segments + index	158.9 ms
Verify one 1,000-message segment	6.9 ms
Materialize 8,192-leaf Merkle tree	448.0 ms
Append 8,192 leaves to compact accumulator	739.8 ms
Kahn order over 8,192 vertices	20.2 ms
Fold 4,096 vertices	106.9 ms

The hashing-heavy Merkle paths deserve batching, WebCrypto call reduction, or a reviewed Worker/WASM backend. The benchmark's roughly 202 MiB process RSS includes Node runtime and transient allocations and must not be interpreted as browser steady-state memory.

7.5 Starting resource profile

The following are conservative experiment defaults, not immutable protocol values:

Parameter	Starting value	Rationale
Snapshot chunk	128 KiB	Resumable without large transient buffers
Active epoch target	4,096 vertices	Keeps fold work near the measured ~100 ms range before device tuning
Hard active cap	8,192 vertices / 8 MiB operations	Bounded join/tail and close work
Direct dependencies	16	Controls fan-out and proof work
Pending memory	10,000 entries and 16 MiB, plus per-peer caps	Operational bound only; eviction is non-semantic
Archive segment	500-1,000 messages or ~1 MiB canonical bytes	Useful paging granularity
Recent archive window	2-8 segments	Product/device dependent
Rollback generations	At least 2 committed epochs	Crash/recovery and bad-release safety
Snapshot fetch concurrency	4-8	Avoids radio/memory contention

Instrumentation should tune these by device class and blueprint. A low-memory mobile profile should close earlier and retain fewer archive segments.

8. Proof-oriented validation

8.1 Snapshot induction

Let `S_e` be the certified state in anchor `A_e`, and `C_e` the exact committed close set. Define:

```
S_(e+1) = Fold(S_e, MinHashKahn(C_e))
```

Assume canonical decode, valid signatures, deterministic authorization against the latched ACL, deterministic reducer/conflict semantics, and a valid commit QC over the descriptor. Every signer computes the same ordered operations and same bytes, so all conforming importers obtain the same `S_(e+1)`. By induction from genesis, importing a certified snapshot is observationally equivalent to archival replay of all committed epochs.

The model checks this over all 5,231 admissible direct-dependency-antichain DAGs through seven vertices and 10,000 randomized epochs.

8.2 Objective stale classification

A valid signature binds `epoch` and `anchor`. After adopting `(currentEpoch, currentAnchor)`, any vertex with an earlier epoch or mismatched same-epoch anchor cannot become a current-epoch vertex without changing signed bytes. Therefore all replicas classify it terminal without historical set membership or delivery-time assumptions.

An unknown dependency with a valid current envelope remains pending because future arrival can complete its cone. This preserves schedule independence.

8.3 Deterministic ordering

Kahn's algorithm with a total hash order chooses the same next vertex from the same zero-in-degree set. Induction over output positions proves identical order. Failure to emit every vertex detects cycles or incomplete dependency closure.

8.4 Quorum intersection and locks

For `q = ceil(2n/3)`, two quorums intersect in at least `2q-n` signers. Under the stated less-than-one-third Byzantine assumption, conflicting same-round QCs require an honest signer to double-vote. Persisted one-vote slots prevent that locally. Cross-round safety additionally depends on lock/valid-value rules and a correct pacemaker; finite intersection arithmetic alone is not a full consensus proof.

The model checks every quorum pair for `n=4..10`, algebraically checks `n=4..10,000`, and checks every same-round QC pair through `n=9`. It also executes the old retroactive-commit schedule and verifies that monotone `enteredRound` prevents the late vote.

8.5 Merkle continuity

RFC 9162's domain-separated tree shape supports logarithmic inclusion and consistency proofs. The JavaScript tests verify every prefix pair through 48 leaves, tamper paths, compare a compact accumulator against materialized roots through 65 leaves, and validate archive descriptor proofs.

8.6 Approximate-membership impossibility at admission

The counterexample is constructive: one replica sees a false-positive “covered” result and permanently rejects a child; another leaves it pending and later accepts it with the valid parent. Therefore any approximate filter with nonzero false-positive probability is unsuitable for semantic terminal classification. It may be used only as an optimization followed by exact verification.

8.7 Crash-safety proof shape

The intended persistent states are:

```
OldActive
  -> ChunksStaged
  -> ManifestAndQCStaged
  -> NewEpochComplete
  -> ActivePointerSwapped
  -> OldGenerationCleanup
```

Only complete staged epochs are pointer targets. Cleanup is not part of commit. A crash before pointer swap returns old active state; after swap it returns new complete state. Production tests must terminate the browser before/after every IndexedDB request and verify this invariant.

9. Validation evidence produced in this review

9.1 Test suite

The reference suite has 22 passing tests covering:

- canonical encoding/decoding and hostile JavaScript values;
- deletion-aware state replacement and context exclusion;
- synchronous deterministic reducers and atomic batch staging;
- Merkle inclusion/consistency proofs and compact roots;
- object/protocol/epoch/anchor domain separation;
- cut/anchor commitment construction;
- stale, pending, and antichain admission;
- bounded non-semantic pending;
- deterministic Kahn order and fail-closed graph validation;
- deterministic pair conflict resolution and cycle detection;
- latched authority and one-boundary fold adoption;
- snapshot chunk/manifest verification;
- archive segment/index proof verification;
- signed value-bound QCs, locks, and retroactive-vote rejection.

Node's experimental coverage report records 91.71% lines, 70.61% branches, and 90.39% functions for the source modules. Browser-only IndexedDB branches are primarily exercised by the included browser harness, not the Node suite.

9.2 Independent epoch model

- 5,231 exhaustive admissible DAGs through seven vertices;
- 10,462 deterministic-order comparisons;
- 5,231 delivery-order comparisons;
- 10,000 randomized epochs;
- 20,000 schedule permutations;
- 132,441 final applied operations;
- 10,000 stale-envelope checks;
- one concrete legacy contraction counterexample reproduced.

9.3 Independent seal model

- 11,612 explicit quorum-pair intersection checks for `n=4..10`;
- 8,547 same-round conflicting-QC pair checks;
- intersection arithmetic for `n=4..10,000`;
- executable late-old-QC fork under the superseded rule;
- fork blocked by monotone round state.

9.4 Static browser audit

Fifteen JavaScript modules, 109,210 source bytes, and 2,624 lines were scanned. The audit reports no Node built-in imports, `Buffer`, `process`, CommonJS `require`, `eval`, synchronous XHR, local/session storage dependency, semantic `Date.now`, semantic randomness, or `JSON.stringify` hashing. Operational timestamps in the journal and cooperative scheduling timers are outside consensus semantics.

9.5 Browser harness limitation

The harness covers WebCrypto hashing, Worker responsiveness, IndexedDB immutable vote-slot races, staged epoch adoption, and recovery. In the supplied execution environment, Chromium returned `ERR_BLOCKED_BY_ADMINISTRATOR` while navigating to the localhost harness because the managed policy blocks all URLs. This is recorded as **blocked**, not passed or failed. The harness must be run unchanged in normal Chrome, Firefox, and Safari profiles during integration.

9.6 Illustrative storage scaling

Assumptions: 512-byte average historical vertex, fixed 8,192-vertex active tail, 10 MiB snapshot, and 100 retained authority changes. Archive payload history is intentionally excluded from compact-peer hot storage.

Lifetime vertices	Full history	AEC v3.1 compact peer	Hard-epoch compact peer
100,000	48.83 MiB	17.05 MiB	14.23 MiB
1,000,000	488.28 MiB	44.52 MiB	14.23 MiB
10,000,000	4.77 GiB	319.18 MiB	14.23 MiB
100,000,000	47.68 GiB	2.99 GiB	14.23 MiB
1,000,000,000	476.84 GiB	29.82 GiB	14.23 MiB

This is an asymptotic model, not measured browser disk use. The hard-epoch result stays flat only because state, active tail, rollback, and authority-change counts are fixed; a product that retains more hot state or local archive segments uses more storage.

10. Repository integration roadmap

Phase 0 - fix current correctness defects

Do not prune yet.

- introduce delete-aware state replacement;
- exclude local context from snapshots;
- make merge/fold batch-atomic;
- type deterministic validation/authorization failures;
- quarantine clock anomalies instead of permanently recording them;
- centralize dependency classification and fail closed on missing ancestry;
- add deterministic Kahn linearizer behind a feature flag;
- add canonical value test vectors and frozen codec;
- add object/protocol domain separation for a **new** object namespace.

Release value: safer current rooms and a trustworthy foundation for snapshots.

Phase 1 - protocol-v2 object namespace

Create new packages or boundaries such as:

```
packages/protocol-v2/       envelopes, codec, hashes, anchors, cuts, votes, QCs
packages/compaction/        close set, fold, snapshot, history accumulator
packages/storage-browser/   IndexedDB journal, vote slots, chunk/archive cache
packages/sync-v2/           epoch summaries, chunk transfer, bounded tail reconciliation
```

Legacy rooms remain on their old hash/topic. Never accept both legacy and new preimages for one object identity.

Phase 2 - deterministic state and shadow snapshots

- integrate canonical snapshot export/import;
- generate snapshots every target epoch without deleting history;
- compare at least two independent replicas and one archive replay;
- add digest mismatch halt-and-alarm telemetry;
- build resumable chunk transfer and quota handling.

Phase 3 - sidecar seal in observation mode

- integrate signer set/authority handoffs;
- implement durable vote slots and Web Locks advisory coordination;
- run prepare/commit/pacemaker without pruning;
- retain full vote/QC traces;
- model protocol formally and replay production traces against the model.

Phase 4 - hard epoch activation

- add epoch/anchor to every new vertex;
- implement close barrier/outbox/rebase UX;
- latch authority;
- reject stale envelopes objectively;
- keep at least two rollback generations and full archives.

Phase 5 - bounded pruning

Only after repeated shadow equivalence and crash-injection success:

- delete closed-epoch active graph structures after durable adoption;
- retain current anchor, bounded tail, rollback generations, authority handoffs, Merkle peaks, and configured archive cache;
- observe memory/disk slopes across long-running browser tests.

Phase 6 - archive paging and product policy

- segment chat history and attachments;
- add verified demand-loading and local search caches;
- expose availability/retention/encryption guarantees in UI;
- add mirror receipts and repair tasks if permanent history is a product promise.

Phase 7 - legacy migration

A legacy creator/current authority signs a migration record naming:

- legacy object ID and chosen terminal legacy frontier/state digest;
- new protocol major and genesis anchor;
- blueprint/schema digests;
- initial authority mode;
- archive/import policy.

Participants explicitly join the new object. A silent in-place hash change would cause legacy peers to mark new vertices invalid and is not viable.

11. Operational and observability requirements

A production deployment should expose, per object:

- active epoch, anchor, target/hard-cap utilization;
- active vertices/bytes and pending entries/bytes by peer;
- close barrier duration and excluded/rebased operation counts;
- prepare/commit/round-change counts, current round, lock state, signer responsiveness;
- fold, encode, hash, chunk, archive, and IndexedDB transaction durations;
- snapshot bytes/chunks and verification failures;
- archive cache hit, proof failure, and unavailable-segment rates;
- storage usage/quota/persistence state;
- rollback generations and recovery outcomes;
- weak-subjectivity tip sources and disagreements;
- digest mismatch and double-vote alarms.

Metrics must not leak message content, private keys, or sensitive ACL data. Signed protocol artifacts should have stable trace IDs derived from their digest.

12. Residual risks and non-negotiable release gates

12.1 Formal consensus model

The reference seal core is not a full pacemaker. A TLA+, Quint, Ivy, or equivalent model must cover message reordering, future-round catch-up, timeout evidence, proposer equivocation, crashes/restarts, lock persistence, signer-set changes, and finalization. Agreement and no-retroactive-vote should be invariants; liveness should be scoped to partial synchrony and quorum responsiveness.

12.2 Real-browser crash matrix

Chrome, Firefox, Safari, iOS, and Android differ in quota, transaction durability, background suspension, Worker lifetime, and eviction. Fault injection must kill tabs/processes at every transition step, including mobile lifecycle events and quota exhaustion.

12.3 Canonical codec review

The production deterministic CBOR profile and schema evolution rules need independent test vectors across JavaScript engines and any non-JS mirror implementation. The bespoke reference codec is executable specification material, not a recommendation to invent a new general codec.

12.4 Blueprint determinism

Application reducers and conflict policies are part of consensus. They require linting/sandboxing rules, deterministic test harnesses, versioned blueprint digests, and migration procedures. Async functions, time, randomness, locale, DOM, network reads, and mutable globals must be unavailable or rejected.

12.5 Availability policy

A certificate proves bytes are correct when found; it does not keep them alive. Permanent Discord-like archives need a policy such as creator plus k mirrors, signed storage receipts, periodic repair, user export, or paid archival services. The protocol should distinguish “certified but unavailable” from “not committed.”

12.6 Freshness and forks

A fresh joiner must compare independent tips or use an invite-pinned checkpoint. Cut-0/authority-handoff equivocation needs clear UX and evidence export. There is no purely local proof that an asynchronous source has shown the latest branch.

12.7 External security review

Before default-on pruning, obtain review of:

- signature/preimage/domain separation;
- authority bootstrap and handoff;
- canonical codec and resource exhaustion;
- lock/pacemaker/vote persistence;
- snapshot/archive proof verification;
- XSS/supply-chain signer compromise;
- migration and rollback behavior.

13. Final recommendation

Adopt the **architecture and sequencing discipline** of AEC v3.1, but replace arbitrary attested cuts with **attested hard epochs** before implementation.

The hard-epoch envelope is what turns history compaction from “delete data and remember every deletion forever” into a bounded protocol. It makes old operations objectively stale, avoids probabilistic or availability-dependent admission, removes synthetic-root tail-order equivalence, and lets compact peers retain state proportional to current semantic state, active work, rollback policy, and authority changes rather than lifetime operation count.

For a browser-first decentralized Discord/game stack, the complete solution is not one snapshot primitive. It is a composition of:

1. a protocol-major authenticated epoch envelope;
2. deterministic, fail-closed replay and latched authority;
3. sidecar attested close consensus with durable browser vote state;
4. canonical chunked snapshots with atomic IndexedDB adoption;
5. append-only audit commitments;
6. application-level archive segmentation for semantically retained chat;
7. bounded active-tail reconciliation;

8. explicit freshness and availability policy;

9. a staged rollout that proves equivalence before pruning.

The included implementation and models establish that this path is technically coherent and remove several high-risk ambiguities from AEC v3.1. They also identify the remaining work honestly. The next engineering milestone should be Phase 0 plus a shadow, non-pruning protocol-v2 snapshot path - not immediate destructive compaction.

Appendices

Appendix A - Legacy ordering counterexample

Dependency list by integer vertex ID:

```
[
  [],
  [0],
  [1],
  [1],
  [0, 2]
]
```

With `{0,1}` baked and `{2,3,4}` retained, the legacy DFS tail order changes from `[2,4,3]` in the original graph to `[2,3,4]` after contraction. Vertex 4 has a redundant related dependency (`0` is an ancestor of 2). This validates both the need for direct-dependency antichains and the recommendation not to retain same-epoch tails across a synthetic-root swap.

Appendix B - Reference artifact map

Path	Purpose
`implementation/docs/attested-hard-epochs-v4.md`	Normative production-target protocol specification
`implementation/src/protocol.js`	Domain-separated vertex/cut/anchor commitments
`implementation/src/admission.js`	Objective admission and bounded pending
`implementation/src/linearize.js`	Kahn order, causality, conflict policies
`implementation/src/state.js`	Deterministic reducer and replacement semantics
`implementation/src/fold.js`	Staged application/ACL fold
`implementation/src/snapshot.js`	Chunk and manifest generation/verification
`implementation/src/archive.js`	Immutable archive segments and Merkle index
`implementation/src/ct-merkle.js`	RFC-style inclusion/consistency and compact accumulator
`implementation/src/seal.js`	Vote/QC/lock/round safety core
`implementation/src/indexeddb-store.js`	Vote CAS and staged epoch adoption
`implementation/browser/`	WebCrypto/Worker/IndexedDB harness
`implementation/model/`	Independent finite/random models and audits
`results/`	Machine-readable test, benchmark, model, and audit evidence

Appendix C - Source references

Repository baseline

- `Faolain/ts-drp-1`, commit `bf7d3516f6ed4be97a755698b4fb3a404e04dc0f`.
- `docs/compaction-attested-epoch-cuts.md` (AEC v3.1).
- `packages/utils/src/hash/index.ts` (`JSON.stringify` vertex hash without object ID).
- `packages/object/src/hashgraph/index.ts` (graph structures and legacy DFS order).
- `packages/object/src/state.ts` (snapshot copy/apply behavior).
- `packages/object/src/drp-applier.ts` (application pipeline, checkpoints, shallow adoption).
- `packages/node/src/operations.ts` and `packages/node/src/handlers.ts` (full-inventory sync).

Supplied Topology research

- *Distributed Real-time Programs* whitepaper, especially snapshot/compaction discussion around pages 13-14.
- *Topology protocol walkthrough*, especially pages 16-19 on snapshotting, compaction, coherence, recursive proofs, and outdated messages.
- AA24 Topology transcript, snapshot/compaction discussion around the extracted transcript's browser-memory and outdated-message section.
- *Practical Rateless IBLT: Part 1* and supplied PRIBLT material, used only for bounded set reconciliation analysis.

Primary standards and protocol references

- RFC 8949, *Concise Binary Object Representation (CBOR)*, deterministic encoding section: https://www.rfc-editor.org/rfc/rfc8949.html
- RFC 9162, *Certificate Transparency Version 2.0*, Merkle tree, inclusion, and consistency proofs: https://www.rfc-editor.org/rfc/rfc9162.html
- W3C, *Indexed Database API 3.0*, transaction durability and atomicity: https://www.w3.org/TR/IndexedDB-3/
- WHATWG, *Storage Standard*, persistence and quota APIs: https://storage.spec.whatwg.org/
- W3C, *Web Locks API*: https://www.w3.org/TR/web-locks/
- W3C, *Web Cryptography Level 2*, CryptoKey storage: https://www.w3.org/TR/webcrypto/
- CometBFT consensus and validator-signing specifications: https://docs.cometbft.com/v0.38/spec/consensus/consensus and https://docs.cometbft.com/v0.38/spec/consensus/signing

Appendix D - Reproduction commands

From the packaged `implementation/` directory:

```
npm test
node --experimental-test-coverage --test test/*.test.mjs
npm run model
npm run bench
npm run browser
```

The browser command requires a normal secure browser profile that permits localhost navigation. The current evidence records the managed-policy block explicitly.